

Technical Solution Document: Batch ETL with AWS Glue and Step Functions for Rental Marketplace Analytics

1. Introduction

1.1 Purpose

This document provides a detailed technical solution for implementing a batch ETL data processing pipeline for a rental marketplace platform. The goal is to extract rental listing and user interaction data from AWS Aurora MySQL, process it using AWS Glue, and load it into Amazon Redshift for business intelligence and analytical reporting.

1.2 Scope

The solution focuses on:

- Extracting data from AWS Aurora MySQL and storing it temporarily in Amazon S3.
- Loading the transformed data into Amazon Redshift.
- Implementing a multi-layered data warehouse architecture (Raw, Curated, and Presentation layers).
- Orchestrating the ETL workflow using AWS Step Functions.

1.3 Definitions, Acronyms, and Abbreviations

- **ETL:** Extract, Transform, Load
- **AWS Glue:** A fully managed ETL service
- **Amazon Redshift:** A data warehouse solution
- **S3:** Amazon Simple Storage Service
- **Aurora MySQL:** A MySQL-compatible relational database

1.4 References

- AWS Glue Documentation: [AWS Glue](#)
- Amazon Redshift Documentation: [Amazon Redshift](#)
- AWS Step Functions Documentation: [Step Functions](#)

1.5 Overview

The document covers the architectural design, key components, data flow, and implementation details of the batch ETL process.

2. Architectural Representation

2.1 Description of Architectural Style and Design Patterns

The solution follows a **layered architecture** with the following components:

- **Source Layer:** AWS Aurora MySQL database.
- **Staging Layer:** Amazon S3 as an intermediate storage layer.
- **Processing Layer:** AWS Glue for transformation and loading.
- **Orchestration Layer:** AWS Step Functions to manage workflow execution.
- **Data Warehouse Layer:** Amazon Redshift for structured analytical queries.

2.2 Architectural Goals and Constraints

- **Scalability:** The solution must handle growing data volumes efficiently.
 - **Performance Optimization:** ETL jobs should be optimized for minimal processing time.
 - **Data Integrity:** Ensure data consistency between Aurora, S3, and Redshift.
 - **Security:** Enforce IAM policies and encryption for data storage.
 - **Cost-Effectiveness:** Minimize AWS service costs through optimal resource allocation.
-

3. Use-Case View

3.1 Use-Case Model

- **Actors:** Data Engineer, BI Analyst, Business User.
- **Use Cases:**
 - Extract rental listing and user activity data.
 - Transform and clean the data.
 - Load data into Redshift for analysis.

- Generate business intelligence reports.

(Placeholder for Use-Case Diagram: Insert a diagram representing use-case interactions.)

3.2 Use-Case Realizations

- Data extraction using AWS Glue crawlers and jobs.
 - Data transformation and validation.
 - Loading processed data into Redshift's multi-layer schema.
 - Query execution for business reporting.
-

4. Logical View

4.1 Major Logical Components

- **ETL Pipeline:** AWS Glue for data processing.
- **Storage:** Amazon S3 for intermediate storage.
- **Data Warehouse:** Amazon Redshift with a multi-layer schema.
- **Orchestration:** AWS Step Functions to manage ETL workflow.

(Placeholder for Class Diagrams and Interaction Diagrams: Add AWS service interactions.)

5. Process View

5.1 Concurrent Processes and Synchronization Mechanisms

- **ETL Execution:** AWS Glue jobs run in parallel for different datasets.
- **Step Functions Coordination:** Ensures sequential execution and error handling.
- **Redshift Data Ingestion:** Parallel copy commands for efficient data loading.

(Placeholder for Process Flow Diagram: Show data flow from Aurora to Redshift.)

6. Deployment View

6.1 Software-to-Hardware Mapping

- AWS Aurora MySQL → Amazon S3 → AWS Glue → Amazon Redshift.

- AWS Step Functions coordinate the execution.

(Placeholder for Deployment Diagram: Illustrate AWS resource interactions.)

7. Implementation View

7.1 Development Environment

- **Programming Language:** Python (for AWS Glue scripts).
- **Tools:** AWS Glue Studio, AWS Step Functions, Redshift Query Editor.
- **Version Control:** AWS CodeCommit / GitHub.

7.2 Configuration Management and Versioning

- Use **AWS Glue versioning** for tracking ETL script changes.
 - Implement **CI/CD pipelines** for ETL updates.
-

8. Data View

8.1 Database Schema

Tables in AWS Aurora MySQL

1. **apartment_attributes** (Stores details of rental listings)
2. **user_viewing** (Tracks user interactions)
3. **apartments** (Basic apartment listings data)
4. **bookings** (Stores booking transactions)

(Placeholder for ER Diagram: Add entity relationships for the above tables.)

8.2 Data Storage and Integrity

- **Data stored in Redshift** using Raw, Curated, and Presentation layers.
 - **Data validation rules** implemented in AWS Glue transformations.
-

9. Size and Performance Considerations

9.1 Performance Benchmarks

- ETL pipeline should complete processing within **X minutes** for a dataset of **Y records**.
- Redshift queries should execute within **Z seconds** for typical analytical queries.

9.2 Scalability Constraints

- Auto-scaling enabled for **AWS Glue workers** based on job size.
 - Amazon Redshift **RA3 nodes** used for scalable compute and storage.
-

10. Quality Attributes

10.1 Key Quality Metrics

- **Usability:** Easy-to-use Redshift tables for BI reporting.
 - **Reliability:** Error handling in Step Functions ensures robust execution.
 - **Performance:** Optimized Glue transformations for minimal processing time.
 - **Maintainability:** Well-documented ETL jobs and version-controlled scripts.
 - **Security:** IAM-based access control and S3 encryption.
-

11. Appendices

11.1 Glossary

(Define key terms like ETL, Redshift, Glue, etc.)

11.2 Index

(Provide an index of key sections.)

11.3 Revision History

Date	Version	Author	Description
YYYY-MM-DD	1.0	Your Name	Initial Draft

Next Steps

- **Diagram Generation:** Use [IT Architecture Diagram Generator](#) for generating use-case, process, and deployment diagrams.
- **Code Implementation:** Develop AWS Glue scripts and Step Functions workflow.
- **Testing and Optimization:** Run ETL jobs and fine-tune performance.

Would you like to add any specific configurations or diagrams to the document? 🚀